

Network and Thread

- 理解客户端、服务器、IP、端口、Socket 的关系
- 能读懂 Java 中 Socket、ServerSocket 的基本通信流程
- 理解阻塞 IO 为什么会引出多线程
- 掌握 Runnable、Thread、start()、run() 的基本关系
- 理解竞态条件、synchronized、对象锁、类锁和死锁

例子：校园聊天室



- 客户端：连接服务器、发送消息、接收消息
- 服务器：监听端口、接收连接、转发消息
- 线程：让“等待消息”和“用户输入”互不打断

网络通信的定位：IP + 端口

- IP 地址：找到哪一台主机（127.0.0.1，本机地址/localhost）
- 端口号：找到这台主机上的哪个程序（0到65536，3306数据库端口）
- Socket：程序和网络之间的通信端点
- 客户端必须知道服务器的 IP 和端口

Socket: 客户端连接服务器

```
Socket socket = new Socket("127.0.0.1", 5000);
```

```
BufferedReader in = new BufferedReader(  
    new InputStreamReader(socket.getInputStream())  
);
```

```
PrintWriter out = new PrintWriter(  
    socket.getOutputStream(), true  
);
```

- new Socket(host, port) 会尝试建立 TCP 连接
- getInputStream() 用来接收服务器数据
- getOutputStream() 用来向服务器发送数据
- 网络代码通常会抛 IOException

发送和接收：本质是读写流

```
// 发送
out.println("大家好, 我是 Alice");

// 接收
String msg = in.readLine();
System.out.println("收到: " + msg);
```

- `println()` 把文本发出去
- `readLine()` 读取一行文本
- 双方要约定消息格式：一条消息一行
- `readLine()` 没读到数据时会阻塞等待

Socket 不是神秘东西。连上以后，程序看到的是输入流和输出流。

ServerSocket: 服务器监听端口

```
ServerSocket server = new ServerSocket(5000);
```

```
System.out.println("服务器启动, 等待连接...");
```

```
Socket client = server.accept();
```

```
System.out.println("有客户端连进来了");
```

- ServerSocket 负责监听一个端口
- accept() 等待客户端连接
- 没有客户端时, accept() 会阻塞
- 连接成功后, 返回一个服务端 Socket

阻塞：网络程序为什么会“卡住等”

常见阻塞点

- `accept()`：没有客户端连接时等待
- `readLine()`：没有新消息时等待
- `sleep()`：主动让线程暂停一段时间
- `wait()`：等待其他线程通知

为什么重要

- 阻塞不是程序错误
- 阻塞表示当前线程暂时无法继续
- 如果只有一个线程，阻塞会让整个功能停住
- 解决思路：把不同任务放到不同线程

单客户端服务器的问题

```
while (true) {  
    Socket client = server.accept();  
    handle(client); // 处理完这个客户端后，才回到 accept()  
}
```

- handle(client) 如果一直读消息，就会阻塞
- 服务器无法及时回到 accept()
- 第二个客户端必须等第一个处理完
- 聊天室显然不能这样工作

为什么聊天室需要多线程

要同时发生的事

- 客户端：用户输入消息
- 客户端：同时接收别人消息
- 服务器：继续接收新客户端
- 服务器：同时处理已连接客户端

线程带来的价值

- 一个线程做一件长期等待的任务
- 接收线程阻塞，不影响发送线程
- 处理 A 客户端，不影响接受 B 客户端
- 多线程让交互更流畅

进程、线程、并发、并行

进程与线程

- 进程：正在运行的程序
- 线程：CPU 调度的基本执行单位
- 一个进程可以包含多个线程
- 同一进程内线程共享部分内存

并发与并行

- 并发：同一时间段内交替推进多个任务
- 并行：同一时刻真正同时执行多个任务
- 单核 CPU 可以并发
- 多核 CPU 才能真正并行

main 线程和新线程



- Java 程序启动时，JVM 会调用 main 方法，形成主线程
- 我们可以在主线程中创建 Thread，让新的执行流程开始运行

Runnable 是任务, Thread 是工人

```
class ReceiveTask implements Runnable {  
    public void run() {  
        while (true) {  
            String msg = in.readLine();  
            System.out.println(msg);  
        }  
    }  
}
```

```
Thread t = new Thread(new ReceiveTask());  
t.start();
```

- Runnable: 描述线程要做的任务
- run(): 任务入口方法
- Thread: 真正负责启动新线程
- start(): 让新线程开始执行 run()

start() 和 run() 不一样

```
Thread t = new Thread(task);
```

```
t.run(); // 普通方法调用, 仍在当前线程执行
```

```
t.start(); // 启动新线程, 由新线程执行 run()
```

- run() 只是普通方法
- 直接调用 run() 不会创建新线程
- start() 才会让 JVM 创建新的调用栈
- 一个 Thread 对象只能 start 一次

客户端：接收消息线程

```
class IncomingReader implements Runnable {  
    private final BufferedReader in;  
  
    IncomingReader(BufferedReader in) {  
        this.in = in;  
    }  
  
    public void run() {  
        try {  
            String msg;  
            while ((msg = in.readLine()) != null) {  
                System.out.println("收到: " + msg);  
            }  
        } catch (IOException e) {  
            System.out.println("连接断开");  
        }  
    }  
}
```

- 这个线程专门等待服务器消息
- readLine() 阻塞也没关系
- 用户输入可以由主线程继续处理
- 客户端因此可以边发边收

服务器：每个客户端一个处理线程

```
ServerSocket server = new ServerSocket(5000);  
  
while (true) {  
    Socket client = server.accept();  
    ClientHandler task = new ClientHandler(client);  
    new Thread(task).start();  
}
```

- 主线程只负责接收新连接
- 每个客户端交给一个 ClientHandler
- 处理线程负责读取该客户端消息
- 主线程能快速回到 accept()

线程生命周期：从创建到结束



- **NEW**: 线程对象创建了, 但还没 start
- **RUNNABLE**: 已经 start, 等待 CPU 调度或正在运行
- **BLOCKED / WAITING / TIMED_WAITING**: 暂时不能继续
- **TERMINATED**: run() 结束或异常终止

线程调度器：不要依赖执行顺序

```
new Thread() -> print("A").start();  
new Thread() -> print("B").start();
```

```
// 可能输出：  
// A B  
// B A  
// A B A B ...
```

- 线程由调度器决定什么时候运行
- 不同机器、不同运行次数，顺序都可能不同
- 程序正确性不能依赖“刚好先执行 A”
- 需要顺序时，要用明确的同步机制

sleep() 不是可靠的顺序控制

```
threadA.start();
```

```
Thread.sleep(1000); // 只是让当前线程睡 1 秒
```

```
threadB.start();
```

- sleep() 只能让当前线程暂停
- 不能保证其他线程一定执行完
- 机器慢、任务多时，等待时间可能不够
- 控制顺序应使用 join、锁、wait/notify 等机制

并发问题：共享数据会被改乱

```
class Counter {  
    int count = 0;  
  
    void addOne() {  
        count = count + 1;  
    }  
}
```

- 两个线程同时调用 addOne()
- `count = count + 1` 不是一个不可分割动作
- 可能两个线程都读到旧值
- 最终少加了一次：更新丢失

银行取款例子：检查和扣款必须连在一起

```
void withdraw(int amount) {  
    if (balance >= amount) {  
        // 线程可能在这里被切走  
        balance = balance - amount;  
    }  
}
```

- 检查余额和扣款不是原子操作
- 两个线程可能都看到余额足够
- 然后都执行扣款
- 结果可能出现余额为负

synchronized: 把关键区域锁起来

```
public synchronized void withdraw(int amount) {  
    if (balance >= amount) {  
        balance = balance - amount;  
    }  
}
```

- 进入 synchronized 方法前要先获得锁
- 同一对象的同一把锁，一次只能被一个线程持有
- 检查余额和扣款变成一个临界区
- 其他线程必须等待锁释放

对象锁：锁的是哪个对象？

```
public synchronized void test1() {  
    // 锁的是 this  
}
```

```
public void test2() {  
    synchronized (this) {  
        // 锁的也是 this  
    }  
}
```

- 非静态 synchronized 方法锁 this
- synchronized(this) 也是锁当前对象
- 同一个对象：竞争同一把锁
- 不同对象：各自有各自的锁

类锁: `static synchronized`

```
public static synchronized void test1() {  
    // 锁的是 SyncDemo.class  
}
```

```
public void test2() {  
    synchronized (SyncDemo.class) {  
        // 锁的也是类对象  
    }  
}
```

- `static synchronized` 锁的是类对象
- 类对象通常写作 `ClassName.class`
- 类锁和对象锁不是同一把锁
- 锁不同, 线程就不会互相阻塞

锁的代价：正确性、效率和风险

锁带来的好处

- 保护共享数据
- 让复合操作具备原子性
- 避免更新丢失
- 让程序结果更可预测

锁带来的代价

- 线程可能等待，效率下降
- 锁范围太大，会降低并发度
- 锁顺序不当，可能死锁
- 调试难度会上升

死锁：互相等，谁也走不了

线程 A 持有锁 1

A 想要 B 的锁 →

线程 B 持有锁 2

等待锁 2

← B 想要 A 的锁

等待锁 1

- 死锁：两个或多个线程互相等对方释放资源，导致程序无法继续执行
- 典型场景：每个线程都拿着一把锁，同时又想要对方手里的锁

如何降低死锁风险

- 固定加锁顺序：所有线程都先拿锁 A，再拿锁 B
- 缩小锁范围：不要持有锁时做耗时 IO 或网络操作
- 避免嵌套锁：能不用两把锁就不用两把锁
- 使用超时机制：拿不到锁时不要无限等待
- 保持锁对象清晰：不要锁字符串常量或不受控对象

聊天室里的共享数据：在线客户端列表

```
class ChatServer {  
    private final List<PrintWriter> clients =  
        new ArrayList<>();  
  
    void addClient(PrintWriter out) {  
        clients.add(out);  
    }  
  
    void broadcast(String msg) {  
        for (PrintWriter out : clients) {  
            out.println(msg);  
        }  
    }  
}
```

- 多个 ClientHandler 线程共享 clients
- 一个线程可能正在遍历
- 另一个线程可能正在添加或删除
- 这就是需要同步保护的地方

同步保护：只锁共享列表操作

```
private final List<PrintWriter> clients = new  
ArrayList<>();
```

```
public void addClient(PrintWriter out) {  
    synchronized (clients) {  
        clients.add(out);  
    }  
}
```

```
public void broadcast(String msg) {  
    synchronized (clients) {  
        for (PrintWriter out : clients) {  
            out.println(msg);  
        }  
    }  
}
```

- 锁的对象是 clients
- 保护 add 和遍历
- 锁范围围绕共享数据
- 真实项目可使用线程安全集合或线程池

网络代码也要处理异常和关闭资源

```
try {  
    Socket socket = new Socket(host, port);  
    // 读写消息  
} catch (IOException e) {  
    System.out.println("网络连接失败");  
} finally {  
    // 关闭 socket / stream  
}
```

- 服务器可能没启动
- 连接可能中途断开
- 读写流可能失败
- Socket 用完要关闭

try-with-resources 管理 Socket

```
try (Socket socket = new Socket("127.0.0.1",  
5000);  
    BufferedReader in = new BufferedReader(  
        new  
InputStreamReader(socket.getInputStream()));  
    PrintWriter out = new PrintWriter(  
        socket.getOutputStream(), true)) {  
  
    out.println("hello");  
    System.out.println(in.readLine());  
}
```

- 资源写在 try 后面括号里
- 代码块结束自动 close
- 异常发生也会 close
- 适合短连接和简单示例

协议：双方要约定消息格式

LOGIN alice

MSG alice 大家好

MSG bob 今天作业是什么？

QUIT alice

- Socket 只负责传字节
- 消息含义由应用层协议决定
- 一行一条消息是最简单协议
- 复杂系统会使用 JSON、HTTP 等格式

一个线程一个客户端的局限

优点

- 优点：模型简单，容易理解
- 每个客户端逻辑清楚
- 适合课堂示例和小规模程序
- 阻塞代码写起来直观

局限

- 客户端很多时，线程数量会很多
- 线程创建和切换都有成本
- 大量阻塞线程会浪费资源
- 真实服务器常用线程池或 NIO

调试网络和线程问题：先问四个问题

- 连不上：IP、端口、服务器是否启动、防火墙是否拦截？
- 发不出：输出流是否 flush，双方协议是否一致？
- 收不到：readLine 是否在等换行，对方是否真的发送？
- 结果乱：是否有共享可变数据，是否需要同步保护？
- 程序卡住：是 accept/readLine 阻塞，还是死锁？

课堂练习：设计迷你聊天室

- 客户端需要几个线程？每个线程负责什么？
- 服务器主线程负责什么？ClientHandler 线程负责什么？
- 在线客户端列表是共享数据吗？哪里需要 synchronized？
- 如果客户端断开连接，服务器应该如何清理资源？
- 如果两个人同时发消息，服务器广播顺序能不能保证？

总结

- Socket 让客户端和服务端通过 IP + 端口建立连接
- ServerSocket 监听端口, accept() 等待客户端连接
- 阻塞 IO 会让线程等待, 因此聊天室天然需要多线程
- Runnable 描述任务, Thread 启动新执行流程, start() 才是真正启动线程
- 多线程访问共享数据会有竞态条件, 需要 synchronized 等同步机制
- 锁能保证正确性, 也会带来性能成本和死锁风险

网络让程序连起来, 线程让程序同时做事, 锁让共享数据不被改乱。